

**EXERCÍCIOS DE INTRODUÇÃO À PROGRAMAÇÃO
EM LÓGICA COM PROLOG - GABARITO**

1)

(a) $\forall x (A(x) \rightarrow (B(x) \rightarrow C(x)))$

$$\forall x (\neg A(x) \vee (B(x) \rightarrow C(x)))$$

$$\forall x (\neg A(x) \vee (\neg B(x) \vee C(x)))$$

$$\forall x (\neg A(x) \vee \neg B(x) \vee C(x))$$

$\neg A(x) \vee \neg B(x) \vee C(x)$ Cláusula Definida (exatamente um literal positivo)

(b) $\forall x (A(x) \vee B(x) \rightarrow C(x))$

$$\forall x (\neg(A(x) \vee B(x)) \vee C(x))$$

$$\forall x ((\neg A(x) \wedge \neg B(x)) \vee C(x))$$

$$\forall x ((\neg A(x) \vee C(x)) \wedge (\neg B(x) \vee C(x)))$$

$(\neg A(x) \vee C(x)) \wedge (\neg B(x) \vee C(x))$ Duas Cláusulas Definidas

(c) $\forall x ((A(x) \rightarrow B(x)) \rightarrow C(x))$

$$\forall x (\neg(A(x) \rightarrow B(x)) \vee C(x))$$

$$\forall x (\neg(\neg A(x) \vee B(x)) \vee C(x))$$

$$\forall x ((\neg\neg A(x) \wedge \neg B(x)) \vee C(x))$$

$$\forall x ((A(x) \wedge \neg B(x)) \vee C(x))$$

$$\forall x ((A(x) \vee C(x)) \wedge (\neg B(x) \vee C(x)))$$

$(A(x) \vee C(x)) \wedge (\neg B(x) \vee C(x))$ Uma das cláusulas geradas não é Definida

(d) $\forall x (\neg A(x) \wedge B(x) \rightarrow \neg C(x))$

$$\forall x (\neg(\neg A(x) \wedge B(x)) \vee \neg C(x))$$

$$\forall x ((\neg\neg A(x) \vee \neg B(x)) \vee \neg C(x))$$

$$\forall x ((A(x) \vee \neg B(x)) \vee \neg C(x))$$

$$\forall x (A(x) \vee \neg B(x) \vee \neg C(x))$$

$A(x) \vee \neg B(x) \vee \neg C(x)$ Cláusula Definida

2) Fórmula: $\forall x (\text{Processo}(x) \wedge \text{Ativo}(x) \rightarrow \exists y \text{Alocado}(y, x))$
 $\forall x (\neg(\text{Processo}(x) \wedge \text{Ativo}(x)) \vee \exists y \text{Alocado}(y, x))$
 $\forall x ((\neg\text{Processo}(x) \vee \neg\text{Ativo}(x)) \vee \exists y \text{Alocado}(y, x))$
 $\forall x (\neg\text{Processo}(x) \vee \neg\text{Ativo}(x) \vee \exists y \text{Alocado}(y, x))$
 $\forall x \exists y (\neg\text{Processo}(x) \vee \neg\text{Ativo}(x) \vee \text{Alocado}(y, x))$
 $\forall x (\neg\text{Processo}(x) \vee \neg\text{Ativo}(x) \vee \text{Alocado}(\text{fskolem}(x), x))$
 $\neg\text{Processo}(x) \vee \neg\text{Ativo}(x) \vee \text{Alocado}(\text{fskolem}(x), x)$
 $\neg(\text{Processo}(x) \wedge \text{Ativo}(x)) \vee \text{Alocado}(\text{fskolem}(x), x)$
 $\text{Processo}(x) \wedge \text{Ativo}(x) \rightarrow \text{Alocado}(\text{fskolem}(x), x)$
 $\text{alocado}(\text{fskolem}(X), X) \text{ :- } \text{processo}(X), \text{ativo}(X).$
(a) $\text{alocado}(\text{fskolem}(X), X) \text{ :- } \text{processo}(X), \text{ativo}(X).$

3) Regra: $\text{alerta}(X) \text{ :- } \text{sensor}(X), \text{leitura_alta}(X, Y).$
 $\forall x \forall y (\text{Sensor}(x) \wedge \text{Leitura_alta}(x, y) \rightarrow \text{Alerta}(x))$

(a) $\forall x \exists y (\text{Sensor}(x) \wedge \text{Leitura_alta}(x, y) \rightarrow \text{Alerta}(x))$

(b) $\forall x (\text{Sensor}(x) \wedge \exists y \text{Leitura_alta}(x, y) \rightarrow \text{Alerta}(x))$

$\forall x (\neg(\text{Sensor}(x) \wedge \exists y \text{Leitura_alta}(x, y)) \vee \text{Alerta}(x))$
 $\forall x ((\neg\text{Sensor}(x) \vee \neg\exists y \text{Leitura_alta}(x, y)) \vee \text{Alerta}(x))$
 $\forall x ((\neg\text{Sensor}(x) \vee \forall y \neg\text{Leitura_alta}(x, y)) \vee \text{Alerta}(x))$
 $\forall x (\neg\text{Sensor}(x) \vee \forall y \neg\text{Leitura_alta}(x, y) \vee \text{Alerta}(x))$
 $\forall x \forall y (\neg\text{Sensor}(x) \vee \neg\text{Leitura_alta}(x, y) \vee \text{Alerta}(x))$
 $\forall x \forall y (\neg(\text{Sensor}(x) \wedge \text{Leitura_alta}(x, y)) \vee \text{Alerta}(x))$
 $\forall x \forall y (\text{Sensor}(x) \wedge \text{Leitura_alta}(x, y) \rightarrow \text{Alerta}(x))$

(c) $\forall x (\text{Sensor}(x) \wedge \forall y \text{Leitura_alta}(x, y) \rightarrow \text{Alerta}(x))$
 $\forall x (\neg(\text{Sensor}(x) \wedge \forall y \text{Leitura_alta}(x, y)) \vee \text{Alerta}(x))$
 $\forall x ((\neg \text{Sensor}(x) \vee \neg \forall y \text{Leitura_alta}(x, y)) \vee \text{Alerta}(x))$
 $\forall x ((\neg \text{Sensor}(x) \vee \exists y \neg \text{Leitura_alta}(x, y)) \vee \text{Alerta}(x))$
 $\forall x (\neg \text{Sensor}(x) \vee \exists y \neg \text{Leitura_alta}(x, y) \vee \text{Alerta}(x))$
 $\forall x \exists y (\neg \text{Sensor}(x) \vee \neg \text{Leitura_alta}(x, y) \vee \text{Alerta}(x))$
 $\forall x \exists y (\neg(\text{Sensor}(x) \wedge \text{Leitura_alta}(x, y)) \vee \text{Alerta}(x))$
 $\forall x \exists y (\text{Sensor}(x) \wedge \text{Leitura_alta}(x, y) \rightarrow \text{Alerta}(x))$

(d) $\exists x \exists y (\text{Sensor}(x) \wedge \text{Leitura_alta}(x, y) \rightarrow \text{Alerta}(x))$

Jamais omita quantificadores universais no meio da fórmula, passe antes para a Forma Normal Prenex.

4) ?- f(bardock, goku).

f(goku, gohan).	f(bardock, goku).	não unifica
f(goku, goten).	f(bardock, goku).	não unifica
f(bardock, goku).	f(bardock, goku).	unifica, resposta: true
f(bardock, raditz).		

(c) true

5) ?- f(goku, X).

f(goku, gohan).	f(goku, X).	unificação possível para X = gohan
f(goku, goten).	f(goku, X).	unificação possível para X = goten
f(bardock, goku).	f(goku, X).	não unifica (a ordem importa)
f(bardock, raditz).	f(goku, X).	não unifica

(d) X = gohan ; X = goten

6) ?- f(raditz, _). Equivale a: “Existe alguém que tem a relação f com raditz?”

f(goku, gohan). f(raditz, _). não unifica

f(goku, goten). f(raditz, _). não unifica

f(bardock, goku). f(raditz, _). não unifica

f(bardock, raditz). f(raditz, _). não unifica (a ordem importa)

A resposta da pergunta é “não”, ou seja, false.

(b) false

7) ?- b(raditz, goku).

b(X, Y) :- b(raditz, goku) :- unifica com X = raditz e Y = goku.

f(Z, X), f(Z, raditz), literal 1

f(Z, Y), f(Z, goku), literal 2

X \= Y. raditz \= goku. literal 3

- Literal 1: f(Z, raditz) unifica com Z = bardock

- Literal 2: f(bardock, goku) unifica com fato, retorna

- Literal 3: raditz \= goku sucesso, retorna true

(d) true

8) ?- u(_ , gohan). Equivale a: “Existe alguém que tem a relação u com gohan?”

u(X, Y) :-	u(_ , gohan) :-	unifica com X = _ , Y = gohan
f(Z, Y),	f(Z, gohan),	literal 1
b(X, Z).	b(_ , Z).	literal 2

- Literal 1: f(Z, gohan) unifica com Z = goku
 f(goku, gohan)

- Literal 2: b(_ , goku)

b(X, Y) :-	b(_ , goku) :-	unifica com X = _ , Y = goku
f(Z, X),	f(Z, _),	literal 4
f(Z, Y),	f(Z, goku),	literal 5
X \= Y.	_ \= goku.	literal 6

- Literal 4: f(Z, _) unifica com Z = goku, _ = gohan
 f(goku, gohan)

- Literal 5: f(goku, goku) não unifica com nada - backtrack

- Literal 4: f(Z, _) unifica com Z = goku, _ = goten
 f(goku, goten)

- Literal 5: f(goku, goku) não unifica com nada - backtrack

- Literal 4: f(Z, _) unifica com Z = bardock, _ = goku
 f(bardock, goku)

- Literal 5: f(bardock, goku) unifica com fato

- Literal 6: `goku \= goku` falha - backtrack
- Literal 4: `f(Z, _)` unifica com `Z = bardock, _ = raditz`
 `f(bardock, raditz)`
- Literal 5: `f(bardock, goku)` unifica com fato
- Literal 6: `raditz \= goku` sucesso, retorna true

`(c) true`

9) ?- g(X, goten).

g(X, Y) :-	g(X, goten) :-	unifica com Y = goten
f(X, Z),	f(X, Z),	literal 1
f(Z, Y).	f(Z, goten).	literal 2

- Literal 1: f(X, Z) unifica com X = goku, Z = gohan
f(goku, gohan).
- Literal 2: f(gohan, goten) não unifica com nada - backtrack
- Literal 1: f(X, Z) unifica com X = goku, Z = goten
f(goku, goten).
- Literal 2: f(goten, goten) não unifica com nada - backtrack
- Literal 1: f(X, Z) unifica com X = bardock, Z = goku
f(bardock, goku).
- Literal 2: f(goku, goten). unifica com Z = Goku, retorna.

(a) X = bardock

10) Consulta: ?- f(3, 4, X).

$f(X, Y, Z) :- Z \text{ is } (X*X + Y*Y)**(0.5).$

$f(3, 4, X) :- X \text{ is } (3*3 + 4*4)**(0.5).$

$f(3, 4, X) :- X \text{ is } (9 + 16)**(0.5).$

$f(3, 4, X) :- X \text{ is } (25)**(0.5).$ (raiz quadrada)

$f(3, 4, X) :- X \text{ is } 5.0.$

(b) $X = 5.0$

11) Consulta: ?- f(X, 4, 5).

$f(X, Y, Z) :- Z \text{ is } (X*X + Y*Y)**(0.5).$

$f(X, 4, 5) :- 5 \text{ is } (X*X + 4*4)**(0.5).$

X está à direita do is e não está instanciada!

(d) Erro

12) Consulta: ?- f(3, 4, 5).

$f(X, Y, Z) :- Z \text{ is } (X*X + Y*Y)**(0.5).$

$f(3, 4, X) :- 5 \text{ is } (3*3 + 4*4)**(0.5).$

$f(3, 4, X) :- 5 \text{ is } (9 + 16)**(0.5).$

$f(3, 4, X) :- 5 \text{ is } (25)**(0.5).$ (raiz quadrada)

$f(3, 4, X) :- 5 \text{ is } 5.0.$ Prolog NÃO unifica inteiro e real

(b) false

13) Consulta: $?- f(6, X)$.

Programa em Prolog para calcular números da Sequência de Fibonacci:

$f(0, 0).$	$F(n) = 0$, se $n = 0$
$f(1, 1).$	$F(n) = 1$, se $n = 1$
$f(X, Y) :-$	$F(n) = F(n - 1) + F(n - 2)$, se $n > 1$
$X > 1,$	
$X1$ is $X - 1,$	
$f(X1, Y1),$	
$X2$ is $X - 2,$	
$f(X2, Y2),$	
Y is $Y1 + Y2.$	$F(6) = 8$

(d) $X = 8$

Note que o X da consulta unifica com o Y da regra.

14) Consulta: $?- p([a, b], X, Y)$.

$p([H T], T, H).$	$p([a, b], X, Y)$	unifica $[H T] = [a, b], X = T, Y = H$
		$[H T] = [a [b]]$
		$H = a, T = [b]$
		$X = [b], Y = a$
		(a) $X = [b], Y = a$

15) Consulta: $?- p(X, [a], b)$.

$p([H T], T, H).$	$p(X, [a], b)$	unifica com $H = b, T = [a],$
		$X = [H T]$
		$X = [b [a]]$
		$X = [b, a]$
		(c) $X = [b, a]$

16) Consulta: $?- f([a, c, b], X)$.

$f([], [])$. $f([a, c, b], X)$. não unifica

$f([H | T], R) :-$ $f([a | [c, b]], X) :-$ unifica $H = a, T = [c, b], R = X$
 $f(T, Rt),$ $f([c, b], Rt),$ literal 1
 $append(Rt, [H], R).$ $append(Rt, [a], X).$ literal 2

- Literal 1: $f([c, b], Rt)$

$f([], [])$. $f([c, b], Rt)$ não unifica

$f([H' | T'], R') :-$ $f([c | [b]], Rt) :-$ unifica $H' = c, T' = [b], R' = Rt$
 $f(T', Rt'),$ $f([b], Rt'),$ literal 3
 $append(Rt', [H'], R').$ $append(Rt', [c], Rt).$ literal 4

- Literal 3: $f([b], Rt')$

$f([], [])$. $f([b], Rt')$ não unifica

$f([H'' | T''], R'') :-$ $f([b | []], Rt') :-$ unifica $H'' = b, T'' = [], R'' = Rt'$
 $f(T'', Rt''),$ $f([], Rt''),$ literal 5
 $append(Rt'', [H''], R'').$ $append(Rt'', [b], Rt').$ literal 6

- Literal 5: $f([], Rt'')$

$f([], []).$ $f([], Rt'')$ unifica com $Rt'' = []$

- Literal 6: `append(Rt'', [ b ], Rt')`
 `append([ ], [ b ], Rt')`
 `Rt' = [ b ]`
- Literal 4: `append(Rt', [ c ], Rt)`
 `append([ b ], [ c ], Rt)`
 `Rt = [ b, c ]`
- Literal 2: `append(Rt, [ a ], X)`
 `append([ b, c ], [ a ], X)`
 `X = [ b, c, a ]`

(a) `X = [ b, c, a ]`

17) Consulta: $\text{?- f}([1, 2, 3], X)$.

$\text{f}([], 0)$. $\text{f}([1, 2, 3], X)$. não unifica

$\text{f}([H T], X) \text{ :-}$	$\text{f}([1 [2, 3]], X) \text{ :-}$	unifica com $H = 1, T = [2, 3]$
$\text{f}(T, X_t),$	$\text{f}([2, 3], X_t),$	literal 1
$X \text{ is } H + X_t.$	$X \text{ is } 1 + X_t.$	literal 2

- Literal 1: $\text{f}([2, 3], X_t)$

$\text{f}([], 0)$. $\text{f}([2, 3], X_t)$ não unifica

$\text{f}([H' T'], X') \text{ :-}$	$\text{f}([2 [3]], X_t) \text{ :-}$	unifica com $H' = 2, T' = [3], X' = X_t$
$\text{f}(T', X_t'),$	$\text{f}([3], X_t'),$	literal 3
$X' \text{ is } H' + X_t'.$	$X_t \text{ is } 2 + X_t'.$	literal 4

- Literal 3: $\text{f}([3], X_t')$

$\text{f}([], 0)$. $\text{f}([3], X_t')$ não unifica

$\text{f}([H'' T''], X'') \text{ :-}$	$\text{f}([3 []], X_t') \text{ :-}$	unifica com $H'' = 3, T'' = [], X'' = X_t'$
$\text{f}(T'', X_t''),$	$\text{f}([3], X_t''),$	literal 5
$X'' \text{ is } H'' + X_t''.$	$X_t' \text{ is } 3 + X_t''.$	literal 6

- Literal 5: $\text{f}([], X_t'')$

$\text{f}([], 0)$. $\text{f}([], X_t'')$ unifica com $X_t'' = 0$

- Literal 6: X_t' is $3 + X_t''$
 X_t' is $3 + 0$
 X_t' is 3 $X_t' = 3$

- Literal 4: X_t is $2 + X_t'$
 X_t is $2 + 3$
 X_t is 5 $X_t = 5$

- Literal 2: X is $1 + X_t$.
 X is $1 + 5$
 X is 6 $X = 6$

(d) $X = 6$